

# FUNÇÕES EM PYTHON

## Material Didático para Ciência da Computação

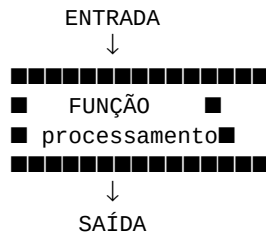
Objetivo: compreender funções como mecanismo de reutilização, modularização, abstração e decomposição de problemas.

Conteúdos: criação e chamada de funções • parâmetros e argumentos • return • escopo • valores padrão • argumentos nomeados • type hints • composição de funções • boas práticas • exercícios e desafio final.

## 1. O que é uma função?

Uma função é um bloco de código criado para executar uma tarefa específica. Em programas maiores, funções ajudam a dividir um problema em partes menores e organizadas.

Uma forma simples de visualizar uma função é como uma máquina: recebe dados de entrada, realiza um processamento e pode produzir uma saída.



Em Ciência da Computação, funções são importantes porque permitem reutilização de código, modularização, abstração, manutenção e testes.

## 2. Por que utilizar funções?

Considere um programa que precisa exibir informações de vários alunos. Sem funções, o mesmo conjunto de instruções pode aparecer repetidamente.

```
print("Aluno: Ana")
print("Curso: Ciência da Computação")
print("Nota: 8.5")
```

```
print("Aluno: João")
print("Curso: Ciência da Computação")
print("Nota: 7.0")
```

Uma função permite transformar essa tarefa em um componente reutilizável.

```
def mostrar_aluno(nome, curso, nota):
    print("Aluno:", nome)
    print("Curso:", curso)
    print("Nota:", nota)

mostrar_aluno("Ana", "Ciência da Computação", 8.5)
mostrar_aluno("João", "Ciência da Computação", 7.0)
```

- Reutilização de código.
- Redução de repetição.
- Melhor organização do programa.
- Facilidade de manutenção.
- Facilidade de testes.
- Separação de responsabilidades.

## 3. Estrutura básica de uma função

```
def nome_da_funcao():
    # instruções
```

A palavra-chave `def` inicia a definição da função. Depois vem o nome da função, os parênteses e dois-pontos. As instruções precisam estar indentadas.

```
def saudacao():
    print("Olá, estudante!")
```

```
saudacao()
```

Na definição, estamos criando a função. Ao escrever `saudacao()`, estamos chamando a função.

## 4. Anatomia de uma função

```
def calcular_soma(a, b):  
    resultado = a + b  
    return resultado
```

- `def`: palavra-chave usada para definir a função.
- `calcular_soma`: nome da função.
- `a, b`: parâmetros.
- `resultado = a + b`: processamento.
- `return`: devolve o resultado para quem chamou a função.

## 5. Função sem parâmetros

```
def apresentar():  
    print("Bem-vindo à Ciência da Computação!")  
  
apresentar()
```

## 6. Função com parâmetros

```
def saudacao(nome):  
    print("Olá, ", nome)  
  
saudacao("Ana")  
saudacao("Carlos")
```

O parâmetro `nome` recebe o valor enviado durante a chamada.

### Parâmetro x argumento

Parâmetro é a variável definida na declaração da função. Argumento é o valor efetivamente enviado na chamada.

```
def saudacao(nome):    # nome = parâmetro  
    print("Olá", nome)  
  
saudacao("Ana")        # "Ana" = argumento
```

## 7. Funções com vários parâmetros

```
def apresentar_aluno(nome, curso, idade):  
    print("Nome:", nome)  
    print("Curso:", curso)  
    print("Idade:", idade)  
  
apresentar_aluno(  
    "Ana",  
    "Ciência da Computação",  
    20  
)
```

## 8. O comando return

O `return` devolve um valor para o código que chamou a função. Isso permite armazenar o resultado, utilizá-lo em expressões ou passá-lo para outra função.

```
def somar(a, b):  
    return a + b  
  
resultado = somar(10, 20)  
print(resultado)
```

Resultado: 30.

print() não é a mesma coisa que return

```
def somar_print(a, b):  
    print(a + b)  
  
resultado = somar_print(10, 20)  
print(resultado)  
  
# Saída:  
# 30  
# None
```

A primeira função apenas exibe o resultado. A segunda, usando return, devolve o valor.

## 9. Funções que retornam diferentes tipos

```
def obter_idade():  
    return 20  
  
def obter_curso():  
    return "Ciência da Computação"  
  
def obter_media():  
    return 8.5  
  
def aprovado(nota):  
    return nota >= 7
```

Uma função pode retornar inteiros, strings, números reais, booleanos, listas, dicionários, objetos e outros valores.

## 10. Funções e condições

```
def verificar_aprovacao(media):  
    if media >= 7:  
        return "Aprovado"  
    else:  
        return "Reprovado"  
  
print(verificar_aprovacao(8))
```

## 11. Composição de funções

Funções podem ser combinadas. Uma função pode produzir um resultado que será utilizado por outra.

```
def calcular_media(n1, n2, n3):  
    return (n1 + n2 + n3) / 3  
  
def verificar_situacao(media):  
    if media >= 7:  
        return "Aprovado"  
    elif media >= 5:  
        return "Recuperação"  
    else:  
        return "Reprovado"
```

```
media = calcular_media(8, 7, 9)
situacao = verificar_situacao(media)

print("Média:", media)
print("Situação:", situacao)
```

Notas  
↓  
calcular\_media()  
↓  
Média  
↓  
verificar\_situacao()  
↓  
Situação

## 12. Funções e decomposição de problemas

Um sistema acadêmico pode ser dividido em várias responsabilidades.

```
SISTEMA ACADÊMICO
■
■■■ cadastrar_aluno()
■■■ listar_alunos()
■■■ calcular_media()
■■■ verificar_aprovacao()
■■■ buscar_aluno()
■■■ remover_aluno()
```

Esse processo é chamado de modularização: o sistema é dividido em módulos ou componentes menores, cada um com uma responsabilidade clara.

## 13. Valores padrão

```
def saudacao(nome="estudante"):
    print("Olá,", nome)

saudacao()
saudacao("Ana")
```

Quando nenhum argumento é fornecido, o parâmetro recebe o valor padrão.

## 14. Argumentos nomeados

```
def apresentar(nome, idade, curso):
    print(nome, idade, curso)

apresentar(
    curso="Ciência da Computação",
    nome="Ana",
    idade=20
)
```

Argumentos nomeados tornam a chamada mais explícita e permitem alterar a ordem dos argumentos.

## 15. Type hints

```
def somar(a: int, b: int) -> int:
    return a + b

def calcular_media(n1: float, n2: float) -> float:
    return (n1 + n2) / 2
```

As anotações de tipo documentam os valores esperados e ajudam editores, ferramentas de análise e outros programadores. Em Python, elas não funcionam como uma barreira rígida de tipos em tempo de execução.

## 16. Escopo de variáveis

Escopo define onde uma variável pode ser acessada.

```
def calcular():
    resultado = 30
    print(resultado)

calcular()

# print(resultado) # erro: resultado é local
```

A variável resultado é local à função. Variáveis globais podem ser acessadas por funções, mas o uso excessivo de estado global dificulta a manutenção e os testes.

## 17. Funções como abstração

Abstração significa esconder detalhes de implementação e fornecer uma interface simples para utilização.

```
media = calcular_media(8, 9)
```

Quem utiliza a função precisa saber quais dados fornecer e qual resultado esperar, mas não precisa repetir os detalhes internos do cálculo.

```
ENTRADA: duas notas
      ↓
    calcular_media()
      ↓
SAÍDA: uma média
```

## 18. Boas práticas

- Use nomes significativos, como calcular\_media() em vez de f().
- Mantenha uma responsabilidade clara por função.
- Evite funções gigantes; divida responsabilidades.
- Evite repetição de código.
- Defina parâmetros e retornos de forma clara.
- Use docstrings quando a função precisar de documentação.
- Evite dependência desnecessária de variáveis globais.

## 19. Exemplo completo — Sistema de notas

```
def calcular_media(n1, n2):
    return (n1 + n2) / 2

def verificar_situacao(media):
    if media >= 7:
        return "Aprovado"
    elif media >= 5:
        return "Recuperação"
    else:
        return "Reprovado"
```

```
def exibir_resultado(nome, media, situacao):
    print("-----")
    print("Aluno:", nome)
    print("Média:", media)
    print("Situação:", situacao)
    print("-----")

nome = input("Nome do aluno: ")
n1 = float(input("Nota 1: "))
n2 = float(input("Nota 2: "))

media = calcular_media(n1, n2)
situacao = verificar_situacao(media)

exibir_resultado(nome, media, situacao)
```

## 20. Atividades práticas

### Exercício 1 — Função simples

Crie uma função chamada `mostrar_mensagem()` que exiba: “Bem-vindo à Ciência da Computação!”

### Exercício 2 — Saudação

Crie uma função `saudacao(nome)` que receba um nome e exiba uma saudação.

### Exercício 3 — Soma

Crie `somar(a, b)` e faça a função retornar a soma. Teste com 15 e 25.

### Exercício 4 — Média

Crie `calcular_media(n1, n2, n3)` e retorne a média das três notas.

### Exercício 5 — Par ou ímpar

Crie `verificar_par(numero)`. A função deve retornar `True` se o número for par e `False` caso contrário.

### Exercício 6 — Maior número

Crie `maior(a, b)` para retornar o maior de dois números. Não utilize `max()`.

### Exercício 7 — Calculadora

Crie quatro funções: `somar()`, `subtrair()`, `multiplicar()` e `dividir()`. Depois crie um programa principal para escolher a operação.

### Exercício 8 — Sistema de notas

Desenvolva um programa com as funções `ler_notas()`, `calcular_media()`, `verificar_situacao()` e `exibir_resultado()`.

### Exercício 9 — Cadastro de alunos

Crie funções `cadastrar_aluno()`, `listar_alunos()` e `buscar_aluno()`. Utilize listas ou dicionários para armazenar os dados.

## 21. Desafio final — Sistema Acadêmico

Desenvolva um sistema utilizando funções, listas ou dicionários, estruturas condicionais, repetição, entrada de dados e return.

```
===== SISTEMA ACADÊMICO =====
```

```
1 - Cadastrar aluno
2 - Listar alunos
3 - Buscar aluno
4 - Calcular média
5 - Verificar situação
6 - Sair
```

Funções mínimas sugeridas:

```
def cadastrar_aluno():
    pass

def listar_alunos():
    pass
```

```
def buscar_aluno():
    pass

def calcular_media():
    pass

def verificar_situacao():
    pass

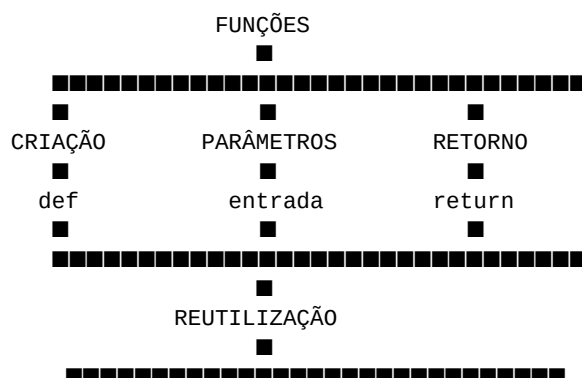
def menu():
    pass
```

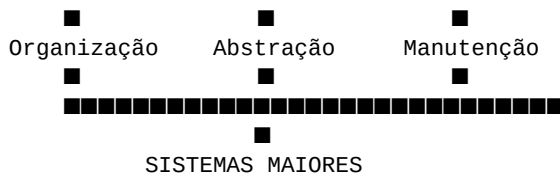
Requisitos: o sistema deve evitar repetição de código, apresentar responsabilidades separadas e possuir nomes de funções significativos.

## 22. Questões de revisão

1. O que é uma função?
2. Qual a diferença entre criar e chamar uma função?
3. O que é um parâmetro?
4. O que é um argumento?
5. Qual a diferença entre print() e return?
6. O que é escopo?
7. O que significa abstração no contexto de funções?
8. Por que devemos evitar funções muito grandes?
9. Qual a vantagem de dividir um sistema em várias funções?
10. Explique como o princípio “dividir para conquistar” pode ser aplicado à programação.

## 23. Mapa mental





## 24. Conceito central

Uma função permite transformar uma tarefa específica em um componente reutilizável do programa.

Para um estudante de Ciência da Computação, o objetivo não é apenas aprender a escrever funções, mas desenvolver a capacidade de decompor problemas, definir responsabilidades, criar abstrações e construir software modular.